QP Code : 25900322

MAHATMA GANDHI UNIVERSITY, KOTTAYAM
MGU-UGP (HONOURS) Regular EXAMINATION October 2025
Third SEMESTER
Discipline Specific Core Course (DSC) - MG3DSCCSC200, MG3DSCECC201
- DATABASE MANAGEMENT SYSTEMS
(2024 ADMISSION ONWARDS)

Duration: 1 Hour 30 Minutes Maximum Marks: 50

Scheme of Valuation

Part A - Very Short Answer Questions
Answer all questions - Each question carries 2 marks

1. Identify the term used to describe the duplication of data Explain different situation in which data duplication occurs [U]/[C01]
   The term used to describe the duplication of data is **Data Redundancy**. (1 mark).
   Explanation of any two situations where data is stored in multiple places (1 mark)
2. State the need of normalization in database. [U]/[CO2]
   Normalization **minimizes data redundancy, ensure data consistency, and improve database efficiency**. (2 marks)
3. Write SQL command to insert a new record into a Student table with attributes ID, Name, and Age. [U]/[CO3]
   Eg, INSERT INTO Student (ID, Name, Age) VALUES (1, 'Joby', 22); (2 marks)
4. Discuss how three-schema architecture supports data abstraction. [U]/[C01]
   The Three-Schema Architecture supports data abstraction by separating the database into three levels—**internal, conceptual, and external**—so that users can access data without worrying about its physical storage or structure. (2 marks)
5. Write the SQL clause to sort a table by column Name in descending order. [K]/[CO3]
   ORDER BY Name DESC; OR
   ORDER BY columnName DESC; (2 marks)

[2 x 5 = 10]

Part B - Short Answer Questions
Answer any 5 questions - Each question carries 5 marks

6. Explain any two functions of DBMS: [K]/[C01]
   A Database Management System (DBMS) provides various functions, including data storage, retrieval, update, and management, ensuring data consistency, security, and integrity. These functions enable efficient data handling, support multiple user access, and facilitate data analysis and decision-making.
   Explanation of any two functions, each 2.5 marks

7. List and explain any two informal guidelines. [K]/[CO2]
   Informal guidelines are not strict rules like normalization, but they provide ways to build efficient database. These guidelines suggest that tables should be designed to minimize data redundancy and dependency, with each table having a single, well-

defined purpose. This includes avoid redundancy in data storage, have atomic columns, avoid storing derived data etc.
List any two guidelines – 2 marks
Explanation of any two guidelines – 3 marks

8. Explain the use of FOREIGN KEY constraint in SQL Write SQL commands to create a STUDENT table with a foreign key referencing DEPARTMENT. [A]/[CO3]

The FOREIGN KEY constraint in SQL is used to link two tables and maintain referential integrity between them. It ensures that the value in one table (child) matches a valid value in another table (parent). For example, each student must belong to a valid department.

```
CREATE TABLE DEPARTMENT (
  DeptID INT PRIMARY KEY,
  DeptName VARCHAR(50)
);

CREATE TABLE STUDENT (
  StudentID INT PRIMARY KEY,
  StudName VARCHAR(40),
  DeptID INT,
  FOREIGN KEY (DeptID) REFERENCES DEPARTMENT(DeptID)
);
```

Explanation of FOREIGN KEY constraint and its usage in SQL - 2.5 marks
Example SQL commands- 2.5 marks

9. Explain the concept of a key attribute and describe how it is identified in a database system. [U]/[C01]

A key attribute is a field or a set of fields in a table that uniquely identifies each record in that table. It helps prevent duplicate entries and ensures accurate data retrieval.

In a DBMS, a key attribute is identified by finding an attribute (or a combination of attributes) that can uniquely distinguish each record in a table. It is usually chosen as the Primary Key because it ensures uniqueness and does not allow null values. Database designers analyze data dependencies and relationships between attributes to determine which one can serve as the key.
Explanation of key attribute – 3 marks
Explanation of how it is identified – 2 marks

10. Compare entity integrity and referential integrity constraints with examples. [An][CO2]

Entity integrity ensures that every table has a unique and non-null primary key, so each record can be uniquely identified. Referential integrity ensures that a foreign key in one table matches a valid primary key value in another table, maintaining consistency between related tables.
For example, in a STUDENT table, StudentID (Primary Key) cannot be null (entity integrity), and in an ENROLLMENT table, StudentID (Foreign Key) must match a valid StudentID in the STUDENT table (referential integrity).
Comparison and explanation – 5 marks

11. SQL queries to analyse employee salaries department-wise: [An]/[CO3]

Sample queries for the three questions are given here. Analyse the answer and give mark accordingly

(i) Find the total salary paid in each department:

```
SELECT Department, SUM(Salary) AS Total_Salary
FROM Employee
GROUP BY Department;
```

(ii) Display only those departments where the average salary is greater than 50,000:

```
SELECT Department, AVG(Salary) AS Average_Salary
FROM Employee
GROUP BY Department
HAVING AVG(Salary) > 50000;
```

(iii) Sort the result in descending order of total salary:

```
SELECT Department, SUM(Salary) AS Total_Salary
FROM Employee
GROUP BY Department
ORDER BY Total_Salary DESC;
```

12. . Demonstrate pattern matching in SQL using LIKE operator [A]/[CO3]

In SQL, the LIKE operator is used for pattern matching in queries to search for specific text patterns within a column and is often combined with the following wildcard characters

% represents any sequence of characters)
_ represents a single character). For example,

Eg: SELECT * FROM Employee WHERE EmpName LIKE 'A%';
retrieves all employees whose names start with the letter A.
Explanation with relevant examples – 5 marks

[5 x 5 = 25]

Part C - Essay Questions

Answer any 1 question - Each question carries 15 marks

13. E-R diagram of a Library Database [K]/[CO1]

The following may be used as a reference for this question.
A library database ER diagram may include entities like Book, Member and Borrowing, connected by relationships. The Book entity has attributes like ISBN, title, and author, while Member has Member ID and name. The Borrowing relationship links a Member to a Book they have borrowed, often with attributes like the borrow date and return date.

Entities and Attributes

- Book: Attributes include ISBN (Primary Key), title, author, and category.
- Member: Attributes include MemberID (Primary Key), name, address, and phone number.

- Borrowing: This is a relationship entity that can have attributes like borrow_date and return_date.

Relationships

- Member "borrows" Book: A member can borrow multiple books, and a book can be borrowed by multiple members, making it a many-to-many relationship (often resolved with the Borrowing entity).
- Borrowing links Member and Book: The Borrowing entity connects a specific member to a specific book, recording the details of that borrowing event.

Example ER Diagram Structure

- Book (Entity)
  - ISBN (Primary Key)
  - title
  - author
- Member (Entity)
  - MemberID (Primary Key)
  - name
- Borrowing (Relationship/Junction Entity)
  - borrow_date
  - return_date
  - MemberID (Foreign Key referencing Member)
  - ISBN (Foreign Key referencing Book)

This structure allows the database to track which member has borrowed which book and when it is due to be returned.

Sample diagram with relevant entities, attributes and relationships (15 marks)

14. Describe in detail the working of nested queries in SQL. Give suitable examples and write SQL queries that use nested queries in SELECT, WHERE, and HAVING clauses [S]/[CO3]

A nested query (or subquery) in SQL is a query inside another query that helps retrieve data based on the results of the inner query. It can appear in the SELECT, WHERE, or HAVING clause.

Example (SELECT):

```
SELECT Name, (SELECT DeptName FROM Department WHERE
Department.DeptID = Employee.DeptID) AS DepartmentName
FROM Employee;
```

Example (WHERE):

```
SELECT Name FROM Employee WHERE DeptID = (SELECT DeptID
FROM Department WHERE DeptName = 'HR');
```

Example (HAVING):

```
SELECT DeptID, AVG(Salary) FROM Employee
GROUP BY DeptID
HAVING AVG(Salary) > (SELECT AVG(Salary) FROM Employee);
```

[15 x 1 = 15]